

Common Lisp

Wade L. Hennessey

Preface

Chapter 1 Introduction

- 1.1 Lisp Usage
- 1.2 A Quick Overview of Lisp's Most Distinguishing Features
- 1.3 Lisp Quasimys
- 1.4 The Joy of Clueless Programming
- 1.5 Lisp History
- 1.6 Overview of This Book

Chapter 2 Getting Started

- 2.1 Playing with Lisp
 - 2.1.1 Evaluation
- 2.2 Symbols, Special Forms and Errors
- 2.3 Lists
- 2.4 Equality and Predicates

Chapter 3 Functions and Recursion

- 3.1 Defining Our Own Functions
 - 3.1.1 The Substitution Model of Function Calls
 - 3.1.2 Side Effects and the Black Box Model of a Function
- 3.2 IF
- 3.3 TRACE
- 3.4 Recursion
 - 3.4.1 The Photocopier Model of Function Calling
 - 3.4.2 Numeric Recursion
 - 3.4.3 List Recursion
 - 3.4.4 Defining APPEND and REVERSE

Chapter 4 A Simple Lisp Program

- 4.1 Program Development

4.2 Example: Peano Arithmetic

4.2.1 Adding Integers

4.2.2 Multiplying Integers

4.3 Giant-Ints

4.3.1 LET Can Name Values

4.3.2 MEMBER

4.3.3 Writing SUCCESSOR for Giant-Ints

Chapter 5 The Versatility of Lists

5.1 List Equality

5.2 Pattern Matching

5.2.1 Association Lists

5.2.2 Implementing MATCH

5.2.3 An Application of MATCH: Program Rewriting

5.3 Lists as Stacks

5.4 Lists as Sets

Chapter 6 Data Abstraction and Structures

6.1 Using Lists to Represent Abstract Data Types

6.2 ERROR and Backtrace

6.3 DEFSTRUCT

6.3.1 SETF Is a Generalized Updating Function

Chapter 7 Destructive Operations

7.1 Memory Organization and Pointers

7.1.1 Building Lists from Cons Cells

7.2 Destructive Operations

7.2.1 Modifying Cons Cells

7.2.2 Defining NREVERSE

7.2.3 Sequencing and Side-Effects

Chapter 8 Programs = Data

8.1 The Advantage of Having Programs = Data

8.1.1 EVAL: Interpreting a Piece of Data as a Program

8.2 Taking Derivatives with DERIV

8.2.1	Improving DERIV with CASE
8.2.2	Improving Derivative Specialists with Backquote
8.3	Data Driven Programming and FUNCALL
8.3.1	Common Lisp Has Separate Function and Value Namespaces
8.3.2	Passing Functions as Arguments
8.4	Evaluating Equations
8.4.1	Using SUBST and EVAL
8.4.2	Evaluating Equations with Bindings
Chapter 9 Consequences of Lexical Scoping	
9.1	Mapping Functions
9.1.1	MAPCAR Is a Collector
9.1.2	MAPCAN Is a Filter
9.2	Lambda List Keywords
9.2.1	&REST Arguments
9.2.2	&OPTIONAL Arguments
9.2.3	&KEY Arguments
9.3	Lexical Scoping and Anonymous Functions
9.3.1	LAMBDA: A Name Is a Terrible Thing to Waste
9.3.2	Lexical Scoping and Functional Objects Interact
9.4	Lexical Closures
9.4.1	Capturing Local State with Lexical Closures
9.4.2	Delaying Evaluation with Lexical Closures
9.5	LABELS and FLET Introduce Lexical Function Definitions
9.6	Some Other Consequences of Lexical Scoping
9.6.1	Using Lexical Functions Correctly
9.6.2	EVAL and the Null Lexical Environment
9.7	Dynamic Scoping
Chapter 10 Syntactic Sugar Makes Lisp Taste Better	
10.1	Iteration
10.1.1	Tail-Recursion: Iteration by Any Other Name
10.1.2	Iteration Viewed as Syntactic Sugar for Tail-Recursion
10.1.3	Tail-Recursion Removal Can Hinder Debugging

- 10.1.4 Frame Walking and Inspection
 - 10.1.5 DO: Yet Another Syntax
 - 10.2 BLOCK and RETURN-FROM Provide Local Control Transfer
 - 10.2.1 Object Lifetimes: BLOCK Names Have Dynamic Extent
 - 10.3 CATCH and THROW Provide Dynamic Control Transfer
 - 10.4 Multiple Values
- Chapter 11** Macros
 - 11.1 Program Rewriting with Macros
 - 11.2 Macro Examples
 - 11.2.1 LET Can Be Written as a Macro
 - 11.2.2 The ITERATE Macro
 - 11.2.3 PROG: Lisp History
 - 11.2.4 SELECT: CASE with Evaluation
 - 11.3 Macro Producing Macros
 - 11.3.1 Nested Backquotes
 - 11.4 Advanced Macro Issues
 - 11.4.1 Argument Destructuring
 - 11.4.2 MACROLET
 - 11.4.3 Macro Restrictions and Problems
- Chapter 12** Arrays, Characters, Strings, and Sequences
 - 12.1 Simple Arrays
 - 12.1.1 Problems with Linked Lists
 - 12.1.2 Using Arrays
 - 12.2 Example: Generating Prime Numbers
 - 12.3 Advanced Array Features
 - 12.3.1 Bit Arrays
 - 12.3.2 Multi-dimensional Arrays
 - 12.3.3 Example: Matrix Multiplication
 - 12.3.4 Array Displacement
 - 12.3.5 Fill Pointers and Adjustable Arrays
 - 12.4 Characters and Strings
 - 12.4.1 Characters Are Objects

12.4.2 Strings Are Vectors of Characters

12.5 Sequence Functions Operate on Lists, Vectors, and Strings

Chapter 13 Input/Output and Tables

13.1 Input and Output Streams

13.1.1 OPEN and Pathnames

13.2 Example: Reading an Index File

13.2.1 UNWIND-PROTECT

13.3 Example: A Binary File Encoder

13.3.1 BYTE Specifiers

13.4 Tables

13.4.1 Hash Tables

13.4.2 Lisp History: Property Lists

Chapter 14 Games and Search

14.1 Tic-Tac-Toe and Minimax

14.1.1 Trees

14.1.2 Game Trees and Board Ranking

14.1.3 Minimax

14.2 Implementing a Tic-Tac-Toe Game Using Minimax

14.2.1 Declarations

14.2.2 A Heuristic Tic-Tac-Toe Board Ranker

14.2.3 Generating and Searching the Game Tree

14.2.4 Putting It All Together

Chapter 15 Missionaries and Cannibals, Backtracking Search

15.1 The Problem: Missionaries and Cannibals

15.2 A Solution to the Problem

15.2.1 A Backtracking Search Algorithm

15.2.2 Data Structures and Declarations

15.2.3 State Manipulation Functions

15.2.4 A Backtracking Search

15.3 Streams

15.3.1 We Can Think of Lists as Eager Streams

- 15.3.2 Changing Our Implementation to Use Streams
- 15.3.3 Lazy Streams

Chapter 16 Data Types and Object-Oriented Programming

- 16.1 The Common Lisp Type Hierarchy
 - 16.1.1 Contagion and Coercion
 - 16.1.2 Extending the Type System
- 16.2 Generic Functions
 - 16.2.1 N-ary Functions
- 16.3 Object-Oriented Programming
 - 16.3.1 Using DEFMETHOD to Define Generic Functions
 - 16.3.2 Generic Function Examples
 - 16.3.3 Message Passing
- 16.4 Implementing Our Object-Oriented System
 - 16.4.1 Representing and Applying Methods
 - 16.4.2 Representing Generic Functions
 - 16.4.3 Defining Methods
- 16.5 Inheritance
 - 16.5.1 Single Inheritance
 - 16.5.2 Multiple Inheritance

Chapter 17 Logic Programming

- 17.1 Imperative versus Declarative Programming
- 17.2 Prolog in Lisp
 - 17.2.1 Facts
 - 17.2.2 Rules
 - 17.2.3 A Declarative Definition of APPEND
- 17.3 Implementing Prolog in Lisp
 - 17.3.1 Unification is Similar to Pattern Matching
 - 17.3.2 Maintaining the Database
 - 17.3.3 Rules and Backtracking
 - 17.3.4 The Inference Engine
- 17.4 Logic Programming Examples
 - 17.4.1 A Simple Natural Language Parser

17.4.2	Prolog Implements Resolution
17.5	Advanced Logic Programming Topics
17.5.1	Procedural Attachment
17.5.2	Search and Efficiency
Chapter 18	A Lisp Interpreter
18.1	Example: An Evaluator with Implicit Control Flow
18.1.1	Environment Manipulation
18.1.2	Problems with Implicit Control
18.2	Continuation Passing Makes Control Flow Explicit
18.2.1	A CPS Version of Factorial
18.2.2	Using CPS to Express Abnormal Control Transfer
18.2.3	Upward Continuations
Chapter 19	Lisp Implementation
19.1	Compiling Lisp Code
19.2	Storing Type Information
19.2.1	Storing the Type in the Object
19.2.2	Tagged Pointers
19.2.3	Big Bag of Pages (BIBOP)
19.3	Dividing Memory into Storage Areas
19.4	Function Call Conventions
19.5	Dynamic Scoping
19.5.1	Shallow Binding
19.5.2	Deep Binding
19.6	Garbage Collection
19.6.1	Stop and Copy
19.6.2	Incremental Stop and Copy
19.6.3	Generation Scavenging
19.6.4	Mark and Sweep
19.6.5	Reference Counting
Chapter 20	Lisp Compilation
20.1	Reducing Common Lisp to the Essentials
20.2	Source Code Rewriting and Alphasizing

- 20.2.1 Single versus Multiple Namespaces
 - 20.2.2 Alpha and Beta Conversion
- 20.3 Using Continuation Passing Style as an Intermediate Language
 - 20.3.1 CPS Makes All Temporary Quantities Explicit
 - 20.3.2 Reordering Argument Evaluation
 - 20.3.3 Function Calls and Tail-Recursion Removal
 - 20.3.4 CPS Makes Multiple Values Simple
- 20.4 Meta-Evaluation and Analysis
 - 20.4.1 Meta-Evaluation
 - 20.4.2 Analysis
- 20.5 Register Allocation and Code Generation
 - 20.5.1 Primitive Operations
 - 20.5.2 Register Allocation
 - 20.5.3 Code Generation
- 20.6 Sample Compilation of **FACT**
- Chapter 21 Efficiency and Large Systems**
 - 21.1 **DECLARE** and Efficiency
 - 21.1.1 Example: Matrix Multiplication
 - 21.1.2 Declaring Variable Types
 - 21.1.3 Speed versus Safety and Time over Space
 - 21.1.4 Adding Declarations to **MATRIX-MULTIPLY**
 - 21.1.5 Additional Declarations
 - 21.2 Garbage Avoidance
 - 21.2.1 Taking Advantage of Dynamic Extent—Stack Allocation
 - 21.2.2 Explicit Resource Allocation and Deallocation
 - 21.3 Packages
 - 21.4 Extensions to Common Lisp
 - 21.4.1 Foreign Function Calls
 - 21.4.2 Error Handling
 - 21.4.3 **LOOP**
 - 21.4.4 Interrupt Handling
 - 21.4.5 Multiprocessing
 - 21.4.6 Parallel Processing

21.4.7 Object-Oriented Programming

21.4.8 Environment Issues

21.5 Delivering Applications

21.6 Conclusion

Bibliography

Index